VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Student Name	Registration Number
SNEHA CHAURASIA	24BCE10791
Slot	**Date of Submission**
B22	31 March 2026

1. Introduction

In today’s world, most of the software we use is built using open-source technology. Instead of being developed by a single company, these tools are created and improved by communities of developers who work together.

Open-source software allows users to use, modify, and share programs freely. Because of this, development becomes faster and collaboration between people becomes easier.

For this project, I have chosen Git as my open-source software. Git is a version control system that helps developers keep track of changes in their code and manage their work efficiently.

In this report, I will discuss the origin, license, Linux usage, and comparison of Git. I have also created five shell scripts to show my practical understanding of Linux commands.

Understanding open-source software is very important today because many modern applications depend on it.

2.Choosing your software

I have selected Git as my open-source software for this project. The system allows developers to track changes in their code and manage individual workflow using microcontrollers.

I chose Git because it is commonly used in the software industry, and exemplifies key concepts of open-source like collaboration and transparency. In the case of this project, it's a good choice because it fits very well into a Linux environment.

3. Project Report

Part A — Origin and Philosophy

A1. The Problem this Software was Created to Solve

Git was created to solve the problem of managing changes in large software projects. Before Git, developers used other version control systems that were often slow, complicated, and difficult to handle, especially when many people were working on the same project.

One big problem with earlier systems was that they were not distributed. Developers had to depend on a central server to store their work. This made collaboration harder and also risky, because if the server failed, the entire project could be affected.

Git was developed by Linus Torvalds in 2005 while he was working on the Linux kernel. He faced several issues with existing proprietary tools, which were not flexible and did not meet his needs. Because of this, he decided to create a system that was fast, reliable, and easy to use.

This was especially important for large teams working on complex projects, where coordination and version tracking were critical.

The main goal of Git was to provide speed, data integrity, and support for distributed development. It allows multiple developers to work independently on the same project without interfering with each other's work. This makes the development process smoother and more efficient.

Another important reason for creating Git was performance. Earlier systems were not efficient when handling large projects with many files and frequent updates. Git was designed to be lightweight and fast, which makes it easier for developers to manage their work without delays.

Git also introduced the idea of distributed version control, where every developer has a complete copy of the project. This reduces dependency on a central server and makes the system more secure and flexible.

Today, Git is one of the most widely used version control systems and plays a very important role in modern software development.

A2. The License — What It Actually Says

Git is released under the GNU General Public License version 2 (GPL v2). This is one of the most well-known licenses in the open-source community. It ensures that the software always remains free and open for everyone to use and improve.

The idea of free software is based on four important freedoms. The first freedom is the ability to run the program for any purpose. The second freedom allows users to study how the program works by accessing its source code. The third freedom is to modify the software according to their needs. The fourth freedom is to share the software with others, either in its original form or after making changes.

This concept of freedom is what makes open-source software different from proprietary software. It gives users more control and flexibility compared to closed systems.

Git follows all these freedoms. Anyone can download it, explore its code, and even make improvements. However, there is one important condition. If someone modifies Git and distributes it, they must also share the modified source code. This ensures that the software always remains open and accessible to everyone.

Because of this license, a company cannot take Git, modify it, and turn it into a closed-source product. If they distribute it, they must keep it open. This is different from licenses like the MIT license, which are more flexible and allow developers to keep their changes private.

Another important concept is the difference between “free as in free beer” and “free as in freedom.” Free software does not always mean that it costs nothing. Instead, it means that users have the freedom to use, modify, and share the software. Git is a good example of this idea.

In conclusion, the GPL license plays an important role in protecting Git and keeping it open for everyone. It allows continuous improvement by developers all over the world.

A3. The Ethics of Open Source

Open source is not only about software and technology, but also about values and ethics. It is based on the idea that knowledge should be shared so that everyone can learn, grow, and benefit from it.

In my opinion, not all software needs to be open source. Some companies spend a lot of time, effort, and money to develop their products, so it is understandable that they may want to protect their work. However, open-source software is still very important because it encourages innovation and helps people learn from existing solutions.

There is also an ethical issue when companies use open-source software to earn profit. Many big companies build their products using open-source tools. This is not wrong, but it can become unfair if they do not give anything back to the community. I believe companies should contribute in some way, such as improving the code, supporting developers, or providing funding.

Open source also promotes transparency, because anyone can see how the software works. This builds trust among users and developers.

The phrase “standing on the shoulders of giants” means building new ideas on top of existing knowledge. This idea is very common in software development. The source makes this process easier because developers can use existing tools and improve them instead of starting from the beginning.

Some people think that open source reduces originality, but I believe the opposite is true. When developers have access to existing code, they can focus more on creating better and more advanced solutions.

In addition, open source also helps students and beginners learn practical skills. By studying real-world projects like Git, they can understand how professional software is built. This makes open source very useful for education.

In the future, open source will continue to play a major role in technology. As more people contribute, software will improve faster and new ideas will develop more easily.

Overall, open source creates a balance between sharing and innovation. It helps developers collaborate, learn from each other, and build better software for the future.

Overall, Part A shows that Git is not just a technical tool but also a strong example of open-source philosophy in action.

Part B — Linux Footprint

Installation Method

On macOS, Git is usually available through Xcode Command Line Tools. It can be installed by running the command `xcode-select --install` in the terminal. After installation, Git can be used directly from the command line without any additional setup. This makes it easy for developers to start using Git quickly.

Key Directories

On macOS, the Git binary is generally located in `/usr/bin/git`. The user-specific configuration file is stored in the home directory as `~/.gitconfig`. In addition, every Git repository contains a hidden `.git` folder. This folder stores all the important data such as version history, branches, and configuration related to that project.

User and Group

Git runs under the currently logged-in user and does not require a separate system user. This is useful from a security point of view because only the user who owns the files can access and modify the repositories. It also keeps things simple for developers working on their own systems.

Service Management

Git does not run as a background service. It is used only when commands are executed in the terminal. Developers interact with Git whenever needed, instead of it running continuously. However, platforms like GitHub provide additional services such as hosting and collaboration features.

Update Model

Git updates on macOS are usually handled through system updates or developer tools. Users can update Git by updating Xcode Command Line Tools or reinstalling Git when needed. The open-source community regularly provides updates, bug fixes, and improvements to ensure that Git remains reliable and efficient.

Screenshot 1: Git Version

```
Last login: Fri Mar 27 10:41:46 on console
snehachaurasia@snehas-MacBook-Air-3 ~ % git --version
git version 2.39.5 (Apple Git-154)
snehachaurasia@snehas-MacBook-Air-3 ~ %
```

Screenshot 2: Script Execution

```
snehachaurasia@snehas-MacBook-Air-3 oss-audit % chmod +x script3.sh
./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: lrwxr-xr-x@ root wheel | Size:   0B
/var/log => Permissions: drwxr-xr-x root wheel | Size:  92M
/Users => Permissions: drwxr-xr-x root admin | Size:  70G
/usr/bin => Permissions: drwxr-xr-x root wheel | Size: 115M
/tmp => Permissions: lrwxr-xr-x@ root wheel | Size:   0B
No global .git directory found
```

Part C — The FOSS Ecosystem

Git is not just a standalone tool; it is part of a much larger open-source ecosystem. It works together with many other tools, platforms, and communities that support software development.

One of the most important parts of Git's ecosystem is platforms like GitHub, GitLab, and Bitbucket. These platforms provide online repositories where developers can store their code, collaborate with others, and track changes efficiently. Without these platforms, Git would be limited to local use only.

Git is also widely used in modern development workflows such as CI/CD (Continuous Integration and Continuous Deployment). Many tools automatically use Git repositories to build, test, and deploy software. This shows how Git acts as a backbone for modern software engineering practices.

Another important connection is with programming languages like Python, Java, and JavaScript. Developers working in these languages use Git to manage their projects and collaborate with teams.

Git also plays a role in the LAMP stack (Linux, Apache, MySQL, PHP), which is widely used for web development. Developers use Git to manage the source code of web applications built using these technologies.

The open-source community behind Git is very strong. Developers from around the world contribute to its improvement by fixing bugs, adding features, and maintaining documentation. This collaborative environment ensures that Git continues to evolve and remain relevant.

Overall, Git is deeply connected with many tools and technologies. It acts as a central system that supports collaboration, version control, and efficient software development across the open-source ecosystem.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source	Proprietary Alternative
Cost	Free to use; no licenses required. Accessible to students, small teams, and startups.	Requires paid licenses or subscriptions; can be expensive for individuals or small organizations.
Security (who can audit the code?)	Fully auditable; anyone can inspect, review, and contribute to the code.	Closed source; only the vendor can audit the code, limiting transparency.
Support and reliability	Supported by a global community; extensive documentation, forums, and tutorials.	Offers official vendor support and enterprise-grade reliability with SLAs.

Freedom to modify	Fully customizable; developers can extend or change the software as needed.	Limited customization; modifications typically require vendor approval or are restricted.
Community vs corporate control	Governed by a global community; contributions and updates come from diverse developers.	Controlled by a single corporation; updates, features, and priorities are driven by business needs.
Your overall verdict	Flexible, cost-effective, widely adopted, and continuously improving through community contributions.	Strong official support and integrated enterprise tools; suitable for large corporations needing structured support

Part E — Conclusion

Recommendation for Real-World Deployment

After studying and using Git, I strongly believe it is an excellent choice for real-world deployment. Because it uses a distributed system, every developer has a full copy of the repository, which makes it very reliable and reduces the risk of losing work. Being open source, Git gives developers the freedom to examine the code, make modifications, and even add new features if needed. Another great advantage is that it's free and widely supported by a large, active community, which makes it accessible to students, small teams, and startups alike. For any project that values teamwork, flexibility, and long-term maintainability, Git provides a powerful combination of technical capability and community-driven support

Contribution and Personal Reflection

I would definitely consider contributing to Git because it offers a unique chance to improve a tool that is used by millions around the world, while also helping me grow as a developer. Contributions could range from updating documentation to fixing issues or creating new features, each of which benefits both the software and the broader community. Although proprietary systems may offer structured support and integrated enterprise features, Git stands out for the freedom it gives developers and the strength of its collaborative community. Working on this project has helped me understand not just the technical aspects of software, but also the value of collaboration, open knowledge sharing, and contributing to tools that can have a global impact.

4. Shell Script Tasks

In this part of the project, I created five shell scripts to explore and audit open-source software using Linux commands. Each script demonstrates a different aspect of shell scripting, such as working with variables, loops, conditionals, file handling, and user interaction. I have provided the script code with detailed comments, an explanation of its purpose, and screenshots of the output when run on a Linux system.

Script 1 — System Identity Report

Code:

```
#!/bin/bash
# ==========================================
# Script 1: System Identity Report
# Author: Sneha Chaurasia
# Course: Open Source Software
#
# Description:
# This script displays system details such as OS,
# kernel version, user info, uptime, and date.
# ==========================================
# -------- Variables (Student Info) --------
STUDENT_NAME="Sneha Chaurasia"
SOFTWARE_CHOICE="Git"
# -------- System Information --------
# Fetch system-related details using commands
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime)
DATE=$(date)
OS=$(sw_vers -productName)
VERSION=$(sw_vers -productVersion)
# -------- Output Section --------
# Display formatted system information
echo "======================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "======================================="
echo "OS      : $OS $VERSION"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Home    : $HOME"
echo "Uptime  : $UPTIME"
echo "Date    : $DATE"
```

echo "License : GNU General Public License (GPL)"

Screenshot:

```
Last login: Sun Mar 29 15:52:02 on ttys000
snehachaurasia@snehas-MacBook-Air-3 ~ % mkdir oss-audit
cd oss-audit
snehachaurasia@snehas-MacBook-Air-3 oss-audit % touch script1.sh script2.sh script3.sh script4.sh script5.sh
snehachaurasia@snehas-MacBook-Air-3 oss-audit % open .
snehachaurasia@snehas-MacBook-Air-3 oss-audit % chmod +x script1.sh
./script1.sh
================================
 Open Source Audit — Sneha Chaurasia
================================
OS      :
Kernel  : 24.2.0
User    : snehachaurasia
Home    : /Users/snehachaurasia
Uptime  : 16:01  up 2 days,  5:21, 2 users, load averages: 1.84 1.83 1.97
Date    : Sun Mar 29 16:01:56 IST 2026
License : GNU General Public License (GPL)
```

Explanation:

In this script, I tried to display some basic information about my system in a simple and clear way. I used different system commands to get details like the operating system, kernel version, current user, uptime, and date. These values are stored in variables and then printed using echo statements.

I also added comments in different parts of the script so that it is easy to understand what each section is doing. This script helped me understand how shell scripting works, especially how to use variables and system commands together to get useful information.

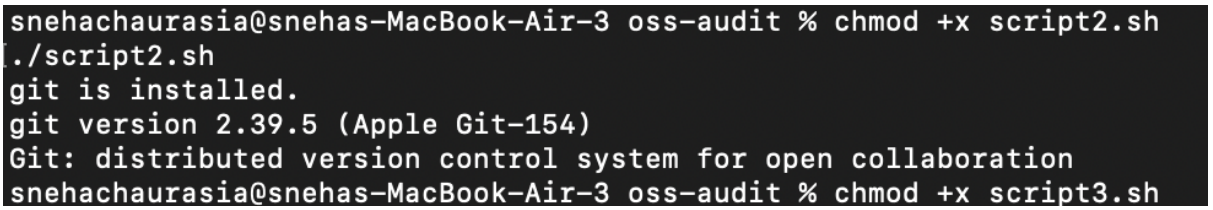
Script 2 — FOSS Package Inspector

Code:

```
#!/bin/bash
# ==========================================
# Script 2: FOSS Package Inspector
# Author: Sneha Chaurasia
# Course: Open Source Software
#
# This script checks if Git is installed,
# displays its version, and gives a short
# description using a case statement.
# ==========================================
# -------- Package Name --------
PACKAGE="git"
# -------- Check Installation --------
```

```
# Using 'command -v' to check if package exists
if command -v $PACKAGE >/dev/null 2>&1; then
   echo "$PACKAGE is installed."
   # Display version of Git
   $PACKAGE --version
else
   echo "$PACKAGE is NOT installed."
fi
# -------- Case Statement --------
# Display description based on package name
case $PACKAGE in
   git)
      echo "Git: A distributed version control system used for collaboration."
      ;;
   apache2)
      echo "Apache: A web server used to host websites."
      ;;
   mysql)
      echo "MySQL: A database system used in many applications.
 ;;
   vlc)
      echo "VLC: A media player that supports multiple formats."
      ;;
   *)
      echo "No description available."
      ;;
esac
```

Screenshot:



```
snehachaurasia@snehas-MacBook-Air-3 oss-audit % chmod +x script2.sh
./script2.sh
git is installed.
git version 2.39.5 (Apple Git-154)
Git: distributed version control system for open collaboration
snehachaurasia@snehas-MacBook-Air-3 oss-audit % chmod +x script3.sh
```

Explanation:

In this script, I checked whether Git is installed on my system using the command “command -v”. If the software is present, the script displays its version using the git --version command. Otherwise, it shows that the software is not installed.

I also used a case statement to print a short description of different software tools. This helped me understand how decision-making works in shell scripting. Through this script, I learned how to check software availability and use conditional statements effectively.

Script 3 — Disk and Permission Auditor

Code:

```
#!/bin/bash
# ==============================================
# Script 3: Disk and Permission Auditor
# Author: Sneha Chaurasia
# Course: Open Source Software
# Description:
# This script checks important system directories
# and displays their permissions, owner, and size.
# It also checks if Git configuration directory exists.
# ==============================================
# -------- Directory List --------
# List of important directories to audit
DIRS=("/etc" "/var/log" "/Users" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "----------------------"
# -------- Loop through directories --------
# Using for loop to process each directory
for DIR in "${DIRS[@]}"; do
    # Check if directory exists
    if [ -d "$DIR" ]; then
        # Get permissions, owner, and group details
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        # Get directory size (suppress errors)
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        # Display directory information
        echo "$DIR => Permissions: $PERMS | Size: $SIZE”
    else
        echo "$DIR does not exist"
    fi
# -------- Git Configuration Check --------
# Check if Git configuration directory exists in home
if [ -d "$HOME/.git" ]; then
    echo "Git config directory exists:"
    # Show permissions of .git directory
    ls -ld $HOME/.git
else
    echo "No global .git directory found"
fi
```

Screenshot:

```
snehachaurasia@snehas-MacBook-Air-3 oss-audit % chmod +x script3.sh
./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: lrwxr-xr-x@ root wheel | Size:   0B
/var/log => Permissions: drwxr-xr-x root wheel | Size:  92M
/Users => Permissions: drwxr-xr-x root admin | Size:  70G
/usr/bin => Permissions: drwxr-xr-x root wheel | Size: 115M
/tmp => Permissions: lrwxr-xr-x@ root wheel | Size:   0B
No global .git directory found
```

Explanation:

In this script, I used a for loop to go through a list of important system directories one by one. For each directory, the script checks whether it exists or not. If it exists, it displays the permissions, owner, and size of that directory.

I used the ls -ld command to get permission details and du -sh to check the size of each directory. I also added a small section to check whether the Git configuration directory exists in the home folder.

This script helped me understand how loops work in shell scripting and how to collect and display system-related information in a structured way.

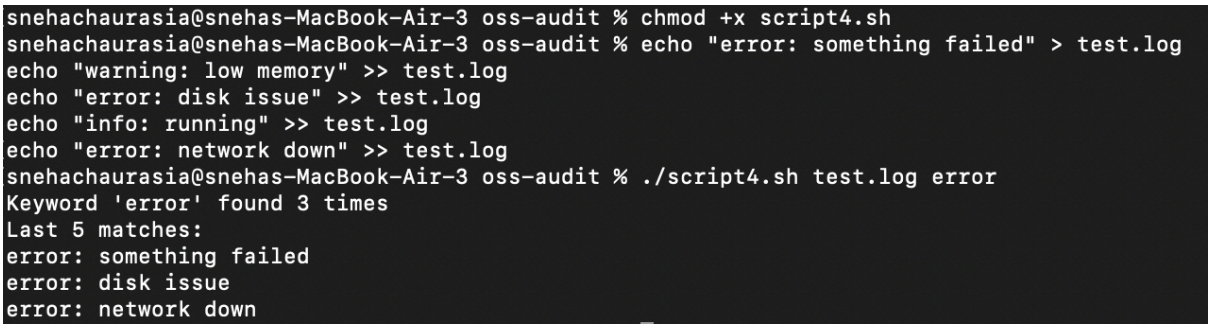
Script 4 — Log File Analyzer

Code:

```
#!/bin/bash
# ==========================================
# Script 4: Log File Analyzer
# Author: Sneha Chaurasia
# Course: Open Source Software
# Description:
# This script reads a log file, counts how many
# lines contain a given keyword, and displays
# the last 5 matching lines.
# ==========================================
# -------- Input Arguments --------
```

```
# First argument: log file name
# Second argument: keyword (default = error)
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
# -------- File Check --------
# Check if the given file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found!"
    exit 1
fi
# -------- Read File Line by Line --------
# Using while loop to process each line
while IFS= read -r LINE; do
    # Check if line contains keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
# -------- Output Section --------
# Display total count of keyword matches
echo "Keyword '$KEYWORD' found $COUNT times"
# -------- Show Last 5 Matches --------
# Display last 5 matching lines
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Screenshot:



Explanation:

In this script, I worked with a log file and tried to find how many times a particular keyword appears in it. The script takes the file name and keyword as input. If no keyword is given, it uses “error” as the default.

I used a while loop to read the file line by line and checked each line using an if condition. Whenever the keyword is found, the counter increases. At the end, the script displays the

total number of matches. I also added a feature to show the last 5 matching lines, which makes the output more useful.

This script helped me understand how to work with files and loops in shell scripting.

Script 5 — The Open Source Manifesto Generator

Code:

```
#!/bin/bash
# ==========================================
# Script 5: Open Source Manifesto Generator
# Author: Sneha Chaurasia
# Course: Open Source Software
# Description:
# This script asks the user three questions and
# generates a personalized open-source manifesto.
# The output is saved in a text file.
# ==========================================
# -------- User Input Section --------
# Asking user for inputs
echo "Answer three questions to generate your manifesto"
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
# -------- File and Date Setup --------
# Get current date
DATE=$(date '+%d %B %Y')
# Output file name using current user
OUTPUT="manifesto_$(whoami).txt"
# -------- Generate Manifesto --------
# Writing personalized message to file
echo "On $DATE, I believe in open source. Using $TOOL daily shows me that freedom
means $FREEDOM. I want to build $BUILD and share it with the world." > $OUTPUT
# -------- Output Section --------
# Display saved file message and content
echo ""
echo "Manifesto saved in $OUTPUT"
cat $OUTPUT
```

Screenshot:

```
snehachaurasia@snehas-MacBook-Air-3 oss-audit % chmod +x script5.sh
snehachaurasia@snehas-MacBook-Air-3 oss-audit % ./script5.sh
Answer three questions to generate your manifesto

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Innovation
3. Name one thing you would build and share freely: Educational platforms

Manifesto saved in manifesto_snehachaurasia.txt
On 31 March 2026, I believe in open source. Using Git daily shows me that freedom means Innovation. I want to build Educational platforms and share it with the world.
```

Explanation:

In this script, I created a small interactive program that asks the user three questions related to open-source software. Based on the answers, the script generates a short personalized manifesto.

I used the read command to take input from the user and stored the responses in variables. Then, I combined these values into a meaningful sentence and saved it into a text file. I also displayed the content of the file at the end.

This script helped me understand how user input works in shell scripting and how to create simple but meaningful outputs using variables and file handling.

Conclusion:

In this project, I got a clear understanding of how open-source software works and why it is so important in today's world. By studying Git, I learned how developers manage their code and work together efficiently on large projects.

While working on this assignment, the shell scripting part was very useful for me. It helped me understand how Linux commands can be used to automate tasks. I learned how to use loops, conditions, variables, and user input in a practical way.

I also understood the importance of sharing knowledge in open-source communities. Developers not only use software but also contribute to improve it, which helps everyone grow together.

Overall, this project helped me connect theory with real-world practice. It improved my technical skills as well as my understanding of open-source philosophy.